

16. do ... while : au moins une fois

Quand il faut d'abord essayer pour savoir si c'est bon, la condition se pose APRÈS.

1. Ce qu'il faut comprendre

`do { ... } while (condition);` exécute le bloc **d'abord**, puis pose la question. Le bloc tourne donc **au moins une fois** — même si la condition est fausse dès le départ. Attention au **point-virgule** final, obligatoire ici.

C'est la boucle du **tirage** : on ne peut pas savoir si un dé est bon avant de l'avoir lancé. Ici, on veut un dé qui **ne donne jamais deux fois de suite** la même face. On relance **tant que** la face est la même que l'ancienne.

Avec un `while` ordinaire, il faudrait tirer une première fois avant la boucle, puis écrire le même tirage dedans : **deux copies** du même code. `do ... while` n'en a qu'une.

`hasard() % 6` donne un reste de 0 à 5 — le modulo du chapitre 1 — et `+ 1` en fait une face de 1 à 6. `essais` compte les relances : parfois 1, parfois 2 ou 3.

Un nouveau lancer toutes les demi-secondes : la boucle `for` du début de tour attend 30 images, comme au compte à rebours.

À toi : empêche aussi le 6 : `while (de == ancien || de == 6)`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **763 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      58 o ← la console, à l'allumage - la mise en route de la console
main()         135 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
Reste          10 o ← main() - le reste d'une division
CopierTuiles   94 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Hasard         25 o ← hasard() - tire un nombre au hasard
Tuiles         304 o ← CopierTuiles - 19 lettres sur 43
"DE:"          3 o ← texte() - un texte gravé
"AVANT:"       6 o ← texte() - un texte gravé
"ESSAIS:"      7 o ← texte() - un texte gravé
```

Lettres gravées : ADEINSTV0123456789:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 3    33 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 3   30 o sur place, dans main() + EcrireNombre (33 o)
```

```
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
hasard() × 1     3 o sur place, dans main() + Hasard (25 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : lance un dé toutes les demi-secondes, sans jamais retomber
// sur la même face que la fois d'avant.
//
// Ce qui est nouveau : do ... while.
// do ... while (condition); fait le bloc AU MOINS UNE FOIS,
// puis le recommence TANT QUE la condition est vraie.
// (Un while normal teste AVANT ; do ... while teste APRÈS.)

#include <texte>      // écrit un texte à l'écran
#include <hasard>    // tire un nombre au hasard
#include <nombre>    // écrit un nombre en chiffres
#include <reste>     // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    uint8_t de = 1;         // La face actuelle (1 à 6).
    uint8_t ancien = 1;     // La face d'avant.
    uint8_t essais = 0;     // Combien de lancers il a fallu cette fois.

    texte(1, 3, "DE:");     // Les trois titres.
    texte(1, 5, "AVANT:");
    texte(1, 7, "ESSAIS:");

    while (true) {          // La boucle du jeu :
        for (uint8_t i = 0; i < 30; i++) image(); // attend 30 images = une demi-seconde.
                                                // (Sans accolades, le for ne répète que image().)

        ancien = de;        // On retient la face d'avant.
        essais = 0;         // On recommence à compter les essais.
        do {                // FAIS (au moins une fois) :
            de = hasard() % 6 + 1; // hasard() rend un nombre au hasard de 0 à 255.
                                // % 6 garde le reste par 6 : 0 à 5 ; + 1 : 1 à 6.
            essais++;        // un essai de plus.
        } while (de == ancien); // la même face : on relance

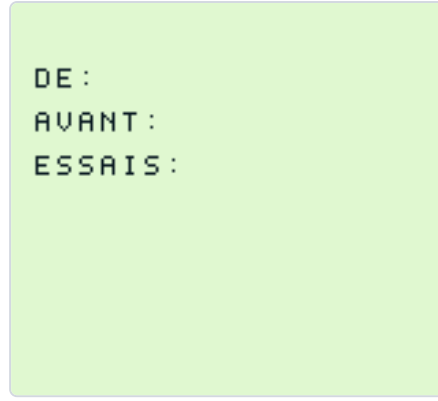
        nombre(9, 3, de, 1); // La nouvelle face.
        nombre(9, 5, ancien, 1); // L'ancienne : jamais la même.
        nombre(9, 7, essais, 1); // Souvent 1, parfois 2 ou plus.
    }
}
```

3. Ce qui se passe, étape par étape

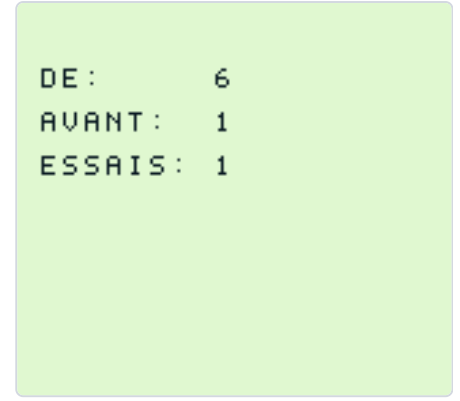
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



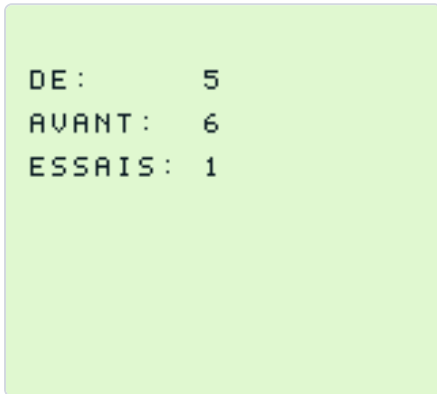
la toute première image — la console vient de démarrer — image 1



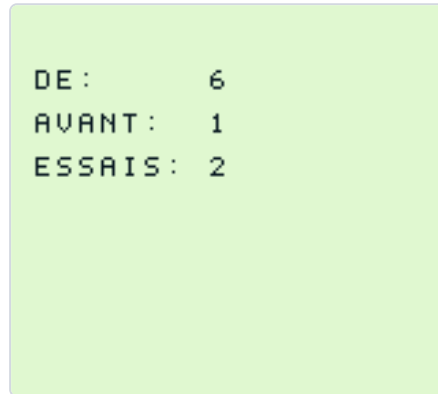
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Une face de 1 à 6 toutes les demi-secondes, jamais la même que la précédente.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ chaque face est entre 1 et 6 (6 1 2 6 5 2 3 5 1 6 4 1)
- ✓ jamais deux fois de suite la même
- ✓ au moins un essai à chaque lancer

← [15. break et continue : sortir, ou sauter un tour](#)

[sommaire](#)

[17. Une boucle qui dure plusieurs images](#) →